Linked List Execution Manual

METHODS, MECHANISMS, AND TERMINAL OUTPUTS

Topic 1: Singly Linked List (class singly)

`insert(5), insert(10)...`

Explanation: Creates a new node with the passed value. If the list is empty, it assigns the node as the head. Otherwise, it traverses to the end of the list (`temp.next != null`) and appends the new node at the tail.

```
Silent execution (State shown in next display)
```

`display()`

Explanation: Traverses the list starting from the head. It prints the `data` of each node and moves to `temp.next` until it reaches a null reference (the end of the list).

```
5 10 15 17 19 20
```

`insertbeg(21)`

Explanation: Instantiates a new node with value `21`. Points the new node's `next` reference to the current `head`, and then updates the `head` pointer to this newly created node, making it the first element.

```
21 5 10 15 17 19 20
```

`insertend(79)`

Explanation: Creates a node with value `79`. Traverses from the head to the last node using a `while` loop. Once at the last node, it sets that node's `next` pointer to the new node.

```
44 22 21 5 10 15 17 19 20 79
```

`sum()`

Explanation: Traverses the list. Uses the modulo operator (`temp.data % 2 == 0`) to check if the data value is even or odd, adding the value to the respective `even` or `odd` integer accumulator variables.

```
oddsum: 156 evensum :214
```

`oddidxevenidxsum()`

Explanation: Maintains an `idx` counter starting at 0. As it traverses the list, it checks if `idx % 2 == 0`. It sums the actual node data into `even` or `odd` accumulators based on the index's parity, not the data's parity.

```
oddindexsum: 206 evenindexsum: 164
```

`insertpos(55, 5)`

Explanation: Navigates to position `pos-1` (4th node) using a `for` loop. It creates a new node with `55`, points its `next` to `temp.next`, and updates the 4th node's `next` to point to the new node, effectively inserting it at the 5th position.

```
44 22 21 5 55 10 15 17 19 20 79 52 66
```

`minmax()`

Explanation: Initializes `min` and `max` variables with the head's data. Traverses the list comparing each node's data against the current min and max, updating them if a smaller or larger value is found.

```
minimum:5, maximux :79
```

`oneafteranother(37, 50) & (37, 55)`

Explanation: Searches for the `afterelement` (50, then 55). If found, it inserts the new node (37) immediately after it by adjusting the `next` pointers. If it reaches the end without finding the element, it prints an error message.

```
linkedlist does not have value: 50 44 22 21 5 55 37 10 15 17 19 20 79 52 66
```

`deletebeg()`

Explanation: Simply updates the `head` pointer to `head.next`. The original head node becomes unreachable and is eventually garbage collected by Java.

```
successfully deleted the head element: 44 22 21 5 55 37 10 15 17 19 20 79 52 66
```

`deleteend()`

Explanation: Traverses to the second-to-last node (`temp.next.next != null`). Once there, it sets `temp.next = null`, detaching the final node from the list.

```
successfully deleted the end element: 66 22 21 5 55 37 10 15 17 19 20 79 52
```

`deletespecificval(17)`

Explanation: Searches for the specific value `17` in the `next` node. When found, it bypasses it by setting `temp.next = temp.next.next`. Also handles edge cases where the value is at the head.

```
successfully deleted the specific value from linked list: 17 22 21 5 55 37 10 15
19 20 79 52
```

`deletebyindex(4)`

Explanation: Iterates while tracking a `pos` counter. When `pos == i-1` (the node right before the target index), it updates the pointer to skip the 4th node (`temp.next = temp.next.next`).

```
successfully deleted the value from 4th place: 55 22 21 5 37 15 19 20 79 52
```

`slllen()`

Explanation: Initializes a counter `c` to 0. Traverses the entire linked list, incrementing `c` for every valid node encountered until `temp` becomes null.

```
SINGLY LINKED LIST LENGTH IS: 9
```

`replace(5, 64)`

Explanation: Searches the list for the target value `5`. Upon finding the first occurrence, it directly overwrites the node's `data` field with the new value `64` and returns immediately.

```
successfully update the value from 5 to 64 22 21 64 37 15 19 20 79 52
```

`search(64) & search(128)`

Explanation: Iterates linearly through the list nodes, comparing `temp.data` to the search argument. Prints success if matched, otherwise completes the loop and prints failure.

```
The value 64 found in list The value 128 not found in list
```

`printmiddle()`

Explanation: Uses the "Tortoise and Hare" algorithm. A `slow` pointer advances by one node, while a `fast` pointer advances by two nodes. When `fast` reaches the end, `slow` is exactly at the middle node.

```
Middle element: 15
```

`reverselist()`

Explanation: Reverses the list iteratively in-place. It uses three pointers (`prev`, `currentnode`, `nextnode`) to flip the `next` reference of each node to point backward, finally setting the `head` to the last processed node (`prev`).

```
reversed list :52 79 20 19 15 37 64 21 22 34
```

`findnode(5)`

Explanation: Traverses the list with a counter `i` starting at 1. When `i == n` (5), it prints the data of that node and exits. Provides array-like index accessing.

```
The element at 5th node from head is 15
```

`findnodefromend(6)`

Explanation: Uses a two-pointer approach. Moves the `first` pointer `n` steps ahead. Then moves both `first` and `second` pointers simultaneously. When `first` hits the end, `second` is exactly `n` nodes from the end.

```
The element found at 6th node from reverse: 15
```

`bubblesort()`

Explanation: Implements a naive bubble sort on the linked list. Uses nested loops to compare adjacent nodes (`temp` and `nn`). If the left value is greater than the right, it swaps their `data` values (not the node objects themselves).

```
Unsorted List: 52 79 20 19 15 37 64 21 22 34 Sorted List: 15 19 20 21 22 34 37 52 64 79
```

`findandremoveduplicate()`

Explanation: Traverses the sorted list. If the current node's data equals the next node's data (`temp.data == temp.next.data`), it updates the pointer to skip the next node (`temp.next = temp.next.next`), removing the duplicate.

```
duplicate value is 22 and removed from list: 15 19 20 21 22 34 37 52 64 79
```

`palindrome()`

Explanation: Finds the middle, reverses the second half, and compares the first half with the reversed second half sequentially. Afterward, it reverses the second half back to restore the original list structure.

```
The list is NOT a palindrome :15 19 20 21 22 34 37 52 64 79
```

`mergesort(s1.head, s2.head)`

Explanation: Merges two pre-sorted lists into a single sorted list. Creates a dummy node and uses a `while` loop to iteratively append the smaller node from `s1` or `s2` to the new list, then attaches any remaining nodes.

```
Merged sorted list: 15 19 20 21 21 22 23 29 33 34 37 44 47 52 56 59 64 77 79 83 93 300
```

Topic 2: Doubly Linked List (class doublyll)

`insertend(12)`

Explanation: Creates a new node. If list is empty, assigns head and tail. Otherwise, sets the new node's `prev` to current `tail`, updates `tail.next` to the new node, and shifts the `tail` pointer to the new node.

```
Doubly list: 12 34 66
```

`displayforward()`

Explanation: Standard traversal starting from `head` and following the `next` pointers until reaching `null`, printing each value.

```
72 84 25 12 34 66
```

`displaybackward()`

Explanation: Starts traversing from the `tail` reference and moves backward by following the `prev` pointers until reaching `null`. Unique to doubly linked lists.

```
Backward doubly list: 66 34 12 25 84 72
```

`insertbeg(25)`

Explanation: Creates a new node. Sets `head.prev` to point to the new node, sets the new node's `next` to the old `head`, and updates the `head` reference to point to the new node.

```
Doubly list: 72 84 25 12 34 66
```

`insertbefore(72, 49)`

Explanation: Finds node with value `72`. Inserts `49` before it by carefully adjusting 4 pointers: `nn.prev = temp.prev`, `nn.next = temp`, `temp.prev.next = nn`, and finally `temp.prev = nn`. Handles head insertion automatically.

```
we successfully inserted the new number 49 before 72 : 49 72 84 25 12 34 66
```

`insertafter(25, 29)`

Explanation: Finds node `25` and inserts `29` after it. Wires the 4 pointers: `nn.next = temp.next`, `temp.next.prev = nn`, `nn.prev = temp`, and `temp.next = nn`. Handles tail insertion explicitly.

```
      we successfully insert the value 29 after the number 25 : 49 72 84 20 25 29 12 34
58 66
```

`deletehead()`

Explanation: Advances the `head` pointer to `head.next`. Disconnects the old head by setting the new head's `prev` pointer to `null`.

```
      we successfully deleted the head 49 : 72 84 20 25 29 11 12 34 58 66 88
```

`deletetail()`

Explanation: Retreats the `tail` pointer to `tail.prev`. Disconnects the old tail by setting the new tail's `next` pointer to `null`.

```
      we successfully deleted the tail 88 : 72 84 20 25 29 11 12 34 58 66
```

`deletepos(6)`

Explanation: Traverses to position `p-1`. Skips the target node by pointing `temp.next = temp.next.next`. If the skipped node isn't null, it also wires backward: `temp.next.prev = temp`.

```
      we successfully deleted the number in the position 6 : 72 84 20 25 29 12 34 58 66
```

`deletevalue(25)`

Explanation: Searches for node with data `25`. When found, links the previous node to the next node (`temp.prev.next = temp.next`) and the next node to the previous (`temp.next.prev = temp.prev`), cleanly extracting it.

```
      we successfully deleted the value 25 in the doubly linked list: 84 20 29 12 34 58
```

`reverse()`

Explanation: Traverses the list swapping the `prev` and `next` pointers of every single node. Once complete, it swaps the global `head` and `tail` pointers to finalize the reversal.

```
reversed list: 95 34 12 29 20 83
```

`findnodefromhead(4)`

Explanation: Simply traverses from the `head` tracking an index counter `i`. When `i == 4`, prints the data.

```
4th node found in list from head: 29
```

`findnodefromtail(3)`

Explanation: Traverses backwards starting from `tail` following `prev` pointers. Tracks an index counter and prints the data when it hits the target index. Highly efficient for finding elements near the end.

```
3th node found in list from tail: 29
```

`search(29)`

Explanation: Linearly checks every node starting from the head. Prints a message immediately upon finding a matching data value.

```
The number 29 found in list :95 34 12 29 20 83
```

`palindrome()`

Explanation: Initializes `left` at head and `right` at tail. Simultaneously moves `left` forward and `right` backward, checking if their data matches, until the pointers cross or meet. Very efficient in doubly linked lists.

```
The list is palindrome: 83 20 29 29 20 83
```

`updatebegposfromendvalpos(3)`

Explanation: Uses one loop to find the 3rd node from the tail to extract its value, then uses a second loop to find the 3rd node from the head and overwrites its data with the extracted value.

```
      we successfully update the value in 3th node from head by using the value from
tail 3th node: 95 34 29 29 20 83
```

`sort()`

Explanation: A naive bubble sort implementation. Uses a double loop to compare `temp1` with `temp2` (subsequent nodes), swapping their integer data fields if they are out of order.

```
      sorted list: 20 20 29 29 83 83
```

`removeallduplicate(29)`

Explanation: Traverses the entire list. Whenever it encounters a node with value `29`, it surgically removes it by bridging the `prev` and `next` nodes around it, properly handling head and tail edge cases.

```
      After removing all duplicate number 29 in list: 20 20 83 83
```

Topic 3: Circular Singly Linked List (class singlycircular)

`insert(10)`

Explanation: Inserts a node. If empty, the node's `next` points to itself (forming a circle). Otherwise, traverses to the last node (`temp.next != head`), links the new node, and sets new node's `next` back to `head`.

```
      circular singly linked list: 10 20
```

`display()`

Explanation: Uses a `do-while` loop. It prints the data and moves `temp = temp.next` continuously until `temp` loops back around and is equal to `head`.

```
10 20
```

`insertbeg(5)`

Explanation: Traverses to the very last node (`temp.next != head`). Inserts the new node at the front, sets its `next` to the old head, updates the global `head` to the new node, and points the last node's `next` to the new head.

```
circular singly linked list: 5 10 20
```

`deletebeg()`

Explanation: Finds the last node in the circular list. Updates `head` to `head.next`. Then wires the last node's `next` pointer to the newly updated head, effectively dropping the first node.

```
The first number in the circular linked list is deleted: 10 20 30
```

`deleteend()`

Explanation: Traverses to the second-to-last node (`temp.next.next != head`). It then sets `temp.next = head`, disconnecting the last node and closing the circle at the new tail.

```
The last number in the circular linked list is deleted: 10 20
```

`deleteval(20)`

Explanation: Searches for the node before the target (`temp.next.data == 20`). Upon finding it, sets `temp.next = temp.next.next` to bypass the deleted node. Handles head/tail deletions by calling the specific methods.

```
New circular singly linked list The value 20 is deleted: 10 30 40 50 60
```

Topic 4: Circular Doubly Linked List (class circulardoublyll)

`insertend(10)`

Explanation: Creates a new node. Sets `nn.next = head` and `nn.prev = tail`. Updates `head.prev = nn` and `tail.next = nn` to maintain the double circular link, and updates the `tail` pointer.

```
circular doubly linked list: 10 20 30
```

`insertbeg(5)`

Explanation: Follows the exact same wiring as `insertend()` to place the node between `tail` and `head`, but shifts the global `head` reference to point to this new node instead of shifting `tail`.

```
circular doubly linked list: 5 10 20 30
```

`displayforward()`

Explanation: Starts at `head` and uses a `do-while` loop, printing data and moving to `temp.next` until `temp` equals `head` again, completing one full forward cycle.

```
display forward: 2 4 5 8 10 20 25 30 35
```

`displaybackward()`

Explanation: Starts at `tail` and uses a `do-while` loop, moving to `temp.prev` until `temp` reaches the `tail` again. The circular links allow endless seamless backward traversal.

```
display backward: 35 30 25 20 10 8 5 4 2
```

`insertbefore(10, 8)`

Explanation: Locates node `10`. Wires `8` before it by adjusting 4 pointers: `nn.prev = temp.prev`, `nn.next = temp`, `temp.prev.next = nn`, and `temp.prev = nn`. Handles head insertion cases.

```
inserted 8 before 10 : 2 5 8 10 20 30
```

`insertafter(20, 25)`

Explanation: Locates node `20`. Inserts `25` after it, safely updating the 4 required doubly-linked pointers. Identical logic to linear doubly, but implicitly handles circular edge cases (like tail wrapping to head).

```
inserted 25 after 20 : 2 4 5 8 10 20 25 30
```

`deletehead()`

Explanation: Moves `head` to `head.next`. Rewires the circular connection by setting the new head's `prev` to `tail` and `tail.next` to the new `head`.

```
deleted head 2 : 4 5 8 10 20 25 30 35
```

`deletetail()`

Explanation: Moves `tail` to `tail.prev`. Closes the circle by setting the new tail's `next` to `head` and the `head.prev` to the new `tail`.

```
deleted tail 35 : 5 8 10 20 25 30
```

`deletepos(3)`

Explanation: Loops until reaching index 3. Extracts the node by pointing the previous node's `next` to the following node, and the following node's `prev` to the previous node.

```
deleted position 3 : 5 8 20 25
```

`deletevalue(10)`

Explanation: Searches for data value `10`. When found, uses `temp.prev.next = temp.next` and `temp.next.prev = temp.prev` to remove it while keeping the circular structure intact.

```
value 10 not in list : 5 20 25
```

`reverse()`

Explanation: Uses a `do-while` loop to visit every node in the circle, swapping its `prev` and `next` pointers. Finally, it swaps the global `head` and `tail` variables to complete the reversal.

```
before reverse: 5 20 25 35 55 95 after reverse: 95 55 35 25 20 5
```

`findnodefromhead(3)`

Explanation: Traverses from `head` while incrementing counter `i`. Since it's circular, the code uses a `do-while` loop to ensure it doesn't spin infinitely if an out-of-bounds index is passed.

```
3th node found in list from head: 5
```

— End of Document —